

One Human, Sixteen Agents: Building Production Software with Role-Specialized LLM Pipelines

Najwa Alghamdi ✉

Independent Researcher / Founder

<https://najwa.io>

Abstract

Most AI-assisted development treats LLMs as tools that help developers write code faster—the developer still architects, reviews, tests, and secures the code themselves. This paper argues for a different framing: LLMs as a *team* that enables engineering processes a solo developer would otherwise routinely skip. We decompose the SDLC into 16 role-specialized agents with enforced tool access, domain-embedded prompts, and structured output schemas, and deploy the pipeline over 11 sprints to build a production multi-tenant SaaS platform. Four findings stand out. *First*, multi-agent role separation enables quality processes—independent review, adversarial testing, dual-perspective validation—that monolithic AI assistance cannot replicate effectively, because a single session playing every role suffers from role contamination, blind spot persistence, and permission overreach. *Second*, runtime tool restriction produces qualitatively different agent behavior: a reviewer with edit access physically removed generates 3–5× longer explanations with file-line references and severity justifications than one merely instructed not to edit—demonstrating that capability restriction, not prompt engineering, enforces role boundaries. *Third*, dual independent reviews using different models (Opus vs. Sonnet+Codex) achieve only 32.8% issue overlap ($n = 125, 8,970$ LOC), with the *cheaper* pairing finding 2× more exclusive issues including all exclusive CRITICALs—suggesting model diversity matters more than model size. *Fourth*, a dedicated red team agent found 8 exploitable vulnerabilities in code that had passed four prior quality stages, including a critical unauthenticated admin API—confirming that adversarial testing catches what standard quality gates systematically miss. None of these gains required a more powerful model; they emerged from process architecture. We introduce cross-model composition (Claude+OpenAI for review, Claude+Gemini for UI validation) and distill seven design principles for practitioners.

2012 ACM Subject Classification Software and its engineering → Software development process management; Computing methodologies → Multi-agent systems; Software and its engineering → Software verification and validation

Keywords and phrases Large Language Models, Multi-Agent Systems, Software Development Life Cycle, Role Specialization, Tool-Use Agents, Code Review Automation, Adversarial Testing, Human-in-the-Loop

Digital Object Identifier 10.4230/LIPIcs...

Category Experience Report

Acknowledgements This paper grew out of a practical need: as an independent researcher and entrepreneur building an AI-native business, I wanted to document what I was learning in real time. I am grateful to Dr. Steve Maddock for his kind review and suggestions, which significantly improved the structure and presentation of this paper.

1 Introduction

1.1 Motivation

Large Language Models have become ubiquitous software development assistants, with tools such as GitHub Copilot, Cursor, and Claude Code seeing rapid adoption since 2023. Yet the dominant interaction paradigm treats AI as a **tool**—a faster way to write code, with a single



© Najwa Alghamdi;
licensed under Creative Commons License CC-BY 4.0

Leibniz International Proceedings in Informatics

LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

LLM session serving identical context, identical permissions, and identical expertise across all phases. The human operator remains solely responsible for architectural decisions, quality judgment, and security assurance. For solo developers and small teams, this means that essential quality gates—independent code review, adversarial security testing, structured QA—are simply skipped. Not because developers are careless, but because one person cannot meaningfully review their own code, attack their own system, or test against acceptance criteria they themselves wrote.

This paper asks a different question: *what if AI served not as a tool that helps you code faster, but as a team that lets you run processes you otherwise couldn't?* Rather than using one powerful model to generate more code, we decompose the SDLC into 16 role-specialized agents—each with constrained capabilities, distinct expertise, and structured outputs—enabling a solo developer to consistently run independent review, adversarial testing, and multi-stage quality gates that would otherwise be routinely skipped.

The monolithic paradigm that we replace suffers from several well-documented limitations:

1. **Role contamination**—An agent that just wrote code is poorly positioned to objectively review it [1]. Cognitive anchoring on one's own implementation reduces the likelihood of identifying architectural issues or alternative approaches.
2. **Blind spot persistence**—A single model's systematic biases apply uniformly across all development phases. If the model tends to overlook certain vulnerability classes, that blind spot persists from design through deployment.
3. **Permission overreach**—An assistant with write access may “helpfully” fix issues rather than articulating them, reducing feedback quality. We demonstrate empirically that this behavioral shift is measurable (Section 5.1).
4. **Domain knowledge dilution**—A generic system prompt cannot encode deep domain-specific checklists, compliance rules, locale-specific business logic, and edge cases simultaneously for all SDLC phases.
5. **Context window saturation**—A monolithic session accumulating design documents, implementation code, review feedback, and test results risks exceeding context limits or degrading reasoning quality through attention dilution.

Human engineering organizations solved these problems long ago through role specialization: architects design, developers implement, reviewers review (with read-only repository access in many organizations), QA engineers test against acceptance criteria, and security teams conduct adversarial assessments. Our thesis is that **process architecture matters more than model capability**—the quality gains in this study come not from using a smarter model, but from structuring how multiple models interact through enforced roles, constrained capabilities, and adversarial stages.

1.2 Research Questions

This study investigates:

1. **RQ1:** Does runtime tool-scoping (enforced capability restriction) produce qualitatively different agent behavior compared to prompt-based instruction alone?
2. **RQ2:** Do dual independent LLM-based code reviews—using different models and augmentation tools—provide complementary or redundant coverage, and does cross-model composition (Opus vs. Sonnet+Codex) yield distinct specialization?
3. **RQ3:** Can an embedded red team agent identify security vulnerabilities that survive the standard design → implement → review → test pipeline?

- 90 4. **RQ4:** What are the practical utilization patterns, failure modes, and cost characteristics
91 of a 16-agent SDLC pipeline over multiple sprints?

92 1.3 Contributions

- 93 1. A detailed architecture for decomposing the SDLC into 16 role-specialized LLM subagents
94 with enforced tool scoping, including a formal tool access matrix and system prompt
95 taxonomy.
- 96 2. Empirical data from 11 production sprints, including dual-review overlap analysis ($n = 125$
97 issues across 8,970 LOC), red team attack results ($n = 30$ attacks), conversation-level
98 test outcomes ($n = 858$ checks), and sprint-by-sprint shipping velocity.
- 99 3. A cross-model review methodology pairing Opus with Sonnet+Codex, revealing that
100 model diversity drives complementary coverage (32.8% overlap)—with Sonnet+Codex
101 excelling at security and Opus at architecture. Additionally, Claude + Gemini composition
102 for UI validation.
- 103 4. A conversation testing framework (**conv-sim**) with 6 personas, 13 scenarios, and 11
104 automated bug detectors for evaluating conversational AI quality.
- 105 5. Seven design principles and four anti-patterns for multi-agent development pipelines,
106 derived from 11 sprints of practitioner experience.

107 2 Related Work

108 2.1 LLMs for Software Engineering

109 Recent work has demonstrated LLM effectiveness across individual SDLC phases: code
110 generation [2, 10], code review [11, 19], test generation [9], and vulnerability detection [15].
111 However, these studies typically evaluate models in isolation on single tasks, rather than as
112 coordinated multi-agent pipelines operating over sustained development periods.

113 SWE-bench [8] and similar benchmarks evaluate LLMs on isolated bug fixes. SWE-
114 agent [22] and OpenHands [20] build agent-computer interfaces that can resolve SWE-bench
115 issues autonomously, while MapCoder [7] decomposes competitive programming into multi-
116 agent pipelines. Liu et al. [12] provide a comprehensive survey of LLM-based agents for
117 software engineering. However, none assess the multi-phase SDLC workflow (design →
118 implement → review → test → secure) that characterizes real software development over
119 sustained periods.

120 2.2 Multi-Agent LLM Systems

121 Multi-agent frameworks such as AutoGen [21], MetaGPT [5], and ChatDev [17] have explored
122 role-based agent decomposition for software tasks. MetaGPT assigns roles (Product Manager,
123 Architect, Engineer) and enforces Standardized Operating Procedures (SOPs) between agents.
124 ChatDev simulates a software company with CEO, CTO, Programmer, and Tester roles.
125 AgentCoder [6] decomposes code generation into programmer, test designer, and test executor
126 agents.

127 Our work differs in four key ways:

- 128 1. **Production deployment**—unlike simulated or benchmark environments, our pipeline
129 was used to build a real, multi-tenant production system over 11 sprints, with shipping
130 deadlines and compliance requirements.

2. **Enforced tool scoping**—prior work relies on prompt-based role adherence; we enforce constraints at the runtime level via tool whitelisting, demonstrating a measurable behavioral difference (Section 5.1).
3. **Adversarial agents**—we include dedicated red team and dual-review agents, absent from prior multi-agent SDLC frameworks.
4. **Cross-model composition**—individual agents combine outputs from multiple LLM providers (Claude + OpenAI for code review; Claude + Gemini for UI validation), extending tool-augmented LLM patterns [18] to cross-provider validation.

2.3 LLM-Based Code Review

Fan et al. [13] surveyed automated code review with LLMs, finding that models can identify functional bugs, security issues, and style violations, but struggle with deep architectural reasoning and domain-specific compliance. Li et al. [11] demonstrated pre-trained models for code review activities, achieving improvements over template-based approaches. Our work extends this by: (a) embedding domain-specific checklists directly into reviewer system prompts; (b) measuring dual-review overlap rates with differentiated review focus; and (c) combining two competing LLM providers for cross-validation.

2.4 Adversarial Testing of LLM Applications

Prompt injection attacks [16, 4] and LLM application vulnerabilities [14] have been extensively documented. Deng et al. [3] proposed systematic red-teaming frameworks for LLM applications. However, embedding adversarial testing as a *standard SDLC phase*—with a dedicated agent that attacks every feature before release, using a structured 7-category attack playbook—is, to our knowledge, novel in practice.

3 System Architecture

3.1 Framework Overview

The system is built on Claude Code’s `.claude/agents/` framework, which allows definition of subagents via markdown files with YAML frontmatter. Each agent definition specifies:

■ **Listing 1** Agent definition schema.

```

---
name: <identifier>
description: <natural language description>
tools: <comma-separated tool whitelist>
model: <opus | sonnet | haiku>
---
<system prompt body: 50-200 lines>

```

The runtime enforces the `tools` whitelist at the infrastructure level: an agent defined with `tools: Read, Glob, Grep, Bash` physically cannot invoke `Edit` or `Write` operations, regardless of what the system prompt instructs or what the model attempts.

3.2 Agent Taxonomy

We define 16 agents organized into a pre-ops pipeline (12 agents) and an ops team (4 agents). Tables 1 and 2 detail the full taxonomy.

Total system prompt volume: ~1,800 lines across 16 agents, comprising approximately 45,000 tokens of domain knowledge, checklists, output templates, and behavioral instructions.

■ **Table 1** Pre-Ops Pipeline Agents (12 agents).

Name	Role	Model	Edit	Web	Lines
Tariq	Product Owner	Opus	–	✓	73
Dana	UX Researcher	Sonnet	–	✓	60
Lina	UI Designer [‡]	Opus	✓	–	144
Faisal	Architect	Opus	–	–	104
Rania	Developer	Opus	✓	–	81
Khaled	Reviewer 1	Opus	–	–	104
Nasser	Reviewer 2 [†]	Sonnet+Codex	–	–	100
Layla	QA Engineer	Opus	✓	–	112
Youssef	Prompt Tester	Opus	✓	–	122
Huda	Conv. Tester	Opus	✓	–	142
Zaid	Red Team	Opus	✓	–	203
Maha	Doc Writer	Sonnet	✓	–	50

[†] OpenAI Codex access via MCP. [‡] Gemini access via MCP.

■ **Table 2** Ops Team Agents (4 agents).

Name	Role	Model	Edit	Lines
Sultan	DevOps	Opus	✓	100
Aws	SRE	Opus	✓	133
Majid	Database Admin	Sonnet	✓	131
Reema	Analytics	Sonnet	✓	143

174 **3.3 Tool Scoping as Access Control**

175 A central design principle is that **tool scoping enforces role boundaries more reliably**
176 **than prompt instruction**. Table 3 illustrates the access control matrix across role
177 categories.

178 The design encodes several deliberate constraints: reviewers cannot edit (forcing articulate,
179 explanatory feedback), architects cannot write code (ensuring output remains at the design
180 level), the Product Owner has web access (for market research), and only 3 roles can spawn
181 child agents.

182 **3.4 System Prompt Architecture**

183 Each agent’s system prompt follows a six-layer structure we formalize as the **IRDCOP**
184 framework: **I**ntity (role, persona), **R**esponsibilities (5–8 duties), **D**omain knowledge
185 (compliance rules, locale logic, edge cases), **C**ross-references (other agents’ outputs), **O**utput
186 format (structured schemas), and **P**inciples (design principles, anti-patterns). Domain
187 rules written once are automatically loaded into every future agent invocation, creating a
188 compounding return on prompt engineering investment.

189 **3.5 Cross-Model Agent Composition**

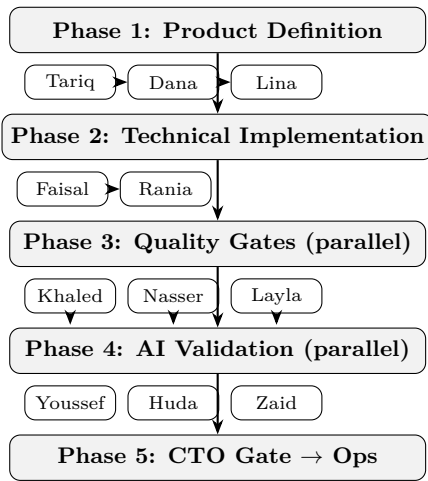
190 Two agents access competing LLM providers via the Model Context Protocol (MCP):

191 **Nasser (Reviewer 2)** is powered by Claude Sonnet but has access to OpenAI Codex
192 via two MCP tools: `codex_review` (sends code to OpenAI for independent review) and
193 `codex_suggest_fix` (sends problematic code for alternative fix suggestions).

194 **Lina (UI Designer)** is powered by Claude Opus but has access to Google Gemini via
195 two MCP tools: `gemini_review_ui` (UI/UX review for accessibility, responsive design, RTL,
196 visual hierarchy) and `gemini_suggest_ui` (alternative component designs).

■ **Table 3** Tool Access Matrix by Role Category.

Capability	PO	Arch	Dev	Rev	QA	Red	Ops
Read files	✓	✓	✓	✓	✓	✓	✓
Search code	✓	✓	✓	✓	✓	✓	✓
Shell exec	✓	✓	✓	✓	✓	✓	✓
Edit files	—	—	✓	—	✓	✓	✓
Write files	—	—	✓	—	✓	✓	✓
Spawn agents	—	✓	✓	—	✓	—	—
Web access	✓	—	—	—	—	—	—
Cross-model	—	—	—	R2	—	—	—



■ **Figure 1** Five-phase pipeline with human orchestrator between phases. Solid arrows = sequential. Dashed arrows = parallelizable.

197 This cross-model composition creates agents that combine the reasoning strengths of
198 multiple providers.

199 **3.6 Pipeline Orchestration**

200 The agents are organized into a five-phase sequential pipeline with a human orchestrator:

201 The human operator serves as orchestrator between phases: invoking agents, passing
202 context, and making gate decisions (“proceed,” “rework,” or “defer”). Within Phases 3 and 4,
203 agents operate independently and can execute in parallel.

204 **3.7 Conversation Testing Framework**

205 We developed a custom conversation testing library (`conv-sim`) that operates as an automated
206 QA harness for conversational AI agents. The framework comprises: **Personas** ($n = 6$) with
207 fixed user IDs, language preferences, and behavioral profiles; **Scenarios** ($n = 13$) covering
208 multi-turn conversations from happy-path to adversarial; **Bug Detectors** ($n = 11$) including
209 language consistency, raw enum leakage, correct agent routing, latency, and others; and a
210 **Reporter** producing terminal summary and JSON reports. All tests execute via real HTTP
211 requests against the running application.

■ **Table 4** Sprint Plan and Deliverables.

Sprint	Focus	Deliverable	Tools
1–2	Foundation	Core platform, Agent A, RAG pipeline, chat UI	19
3–6	Agent B	Multi-step workflows, cross-agent routing, security	14
7	Agent C M1	Domain pipeline, AI-assisted content, web search	13
8	Agent C M2	Interactive AI workflows, structured assessments	8
9	Agent C M3	Cross-module analysis and decision support	7
10	Agent D	Operational intelligence, compliance monitoring	17
11	Agent E	Executive analytics and compliance dashboards	7

212

4Methodology

213

4.1Study Context

214

215

216

217

218

219

The pipeline was used to build a production multi-tenant B2B SaaS platform targeting Saudi Arabia. The platform serves multiple conversational AI agents behind a shared chat interface, each operating over the same multi-tenant data layer. The domain involves structured workflows, government compliance rules, and bilingual user interactions. The technical profile includes: Python 3.12/FastAPI, PostgreSQL+pgvector, row-level tenant isolation, bilingual Arabic/English support, 22 tables, 123 API endpoints, ~40K LOC, and 126 tests.

220

4.2Study Duration and Sprint Structure

221

222

223

Eleven development sprints were conducted between February and March 2026 by a single developer. Each sprint followed the five-phase pipeline (Figure 1). Table 4 summarizes sprint deliverables.

224

4.3Data Collection

225

226

227

228

229

We collected five categories of data: (1) agent invocation logs (which agents, frequency, output size, session duration); (2) code review findings (issues by severity, type, and reviewer); (3) red team results (attack outcomes, severity, post-fix verification); (4) conversation test results (`conv-sim` JSON reports with per-turn, per-detector pass/fail); and (5) shipping metrics (tools per agent, sprints, pipeline passes, review rounds).

230

4.4Dual-Review Protocol

231

232

233

234

235

236

237

For each major feature, both Khaled (Reviewer 1) and Nasser (Reviewer 2) independently review the same code. The protocol enforces: blind independence (Nasser does not see Khaled’s output), differentiated focus (Khaled emphasizes architecture and performance; Nasser emphasizes edge cases, race conditions, and Arabic text handling), cross-model validation (Nasser consults OpenAI Codex via MCP), and structured output (tabular issue listings with file:line references and severity classifications). Issues are classified post-hoc as found by Khaled only, Nasser only, or both.

■ **Table 5** Monolithic-Style vs. Role-Separated Agent Comparison.

Metric	Monolithic	Role-Separated
Edit attempts	~15%	0%
Avg sentences per issue	1–2	3–5
Includes file:line ref	~60%	~95%
Includes “why it matters”	~30%	~85%
Structured output compliance	Variable	Consistent
Inline patches (vs. explanation)	Common	Never

4.5 Red Team Protocol

Zaid (Red Team agent) operates with a 203-line system prompt containing a structured attack playbook across 7 categories: prompt injection (5 subcategories), data exfiltration (3), privilege escalation (3), tenant boundary violation (2), PII leakage, API security (3), and abuse scenarios (3). The red team agent is invoked *only after* the standard pipeline is complete, simulating an external attacker.

5 Results

The core question is whether a multi-agent pipeline with enforced role separation produces measurably different outcomes than monolithic AI assistance. The results address this across four dimensions.

5.1 RQ1: Monolithic vs. Role-Separated Agent Behavior

We tested the same reviewer model under two configurations: (a) **monolithic-style**—all tools available, with a prompt instruction to “review only, do not edit files”; and (b) **role-separated**—Edit/Write tools physically removed. Table 5 summarizes the results.

The monolithic-style agent attempted file edits in ~15% of reviews, violating its role boundary despite explicit instruction. When it produced review-only output, descriptions were terse (1–2 sentences), often providing inline code patches rather than explaining the underlying problem. The role-separated agent produced qualitatively different output—not just the same review with fewer edits, but fundamentally more explanatory, structured, and useful feedback. We term this mechanism **behavioral redirection**: when an agent cannot act on problems directly, it redirects cognitive effort into articulation.

Finding: Runtime capability restriction produces qualitatively different—and higher quality—review output than prompt-based instruction alone.

5.2 RQ2: Dual Review Coverage and Cross-Model Composition

Both reviewers independently reviewed all 5 production agent files (~8,970 LOC, 85 tools). Table 6 shows the aggregate results.

Overlap rate: 32.8%. The combination catches 80% more issues than either reviewer alone. Sonnet+Codex found 2× more exclusive issues (56 vs. 28), and all 5 exclusive CRITICAL findings. Table 7 reveals severity distribution.

Opus excelled at architectural patterns (DRY violations, N+1 detection, design compliance). Sonnet+Codex excelled at security boundary analysis (race conditions, cross-tenant isolation gaps, prompt injection vectors) and domain knowledge conflicts.

Finding: Dual independent reviews with differentiated prompts and models produce strongly complementary coverage (32.8% overlap). Model diversity matters more than model size.

■ **Table 6** Dual-Review Issue Classification ($n = 125$, 5 agent files).

Classification	Count	%
Found by Khaled only (Opus)	28	22.4
Found by Nasser only (Sonnet+Codex)	56	44.8
Found by both (overlap)	41	32.8
Total unique issues	125	100

■ **Table 7** Model Performance: Exclusive Finds by Severity.

Severity	Opus only	Sonnet+Codex only	Both
CRITICAL	0	5	4
HIGH	1	12	9
MEDIUM	7	18	16
LOW	12	14	9
INFO	8	7	3
Total	28	56	41

272 **5.3 RQ3: Red Team Effectiveness**

273 Table 8 shows results across 6 red team sessions.

274 **8 of 30 attacks (26.7%) succeeded** against code that had passed all prior quality
275 gates. The most critical finding: admin API endpoints with zero authentication—full CRUD
276 on sensitive records, accessible by anyone. This vulnerability survived architecture design,
277 implementation, dual code review, and QA testing. Only the adversarial red team agent
278 identified it.

279 *Finding: Standard SDLC quality gates have systematic blind spots for authorization-layer*
280 *vulnerabilities. Dedicated adversarial testing adds non-redundant value (26.7% attack success*
281 *rate against “finished” code).*

282 **5.4 RQ4: Utilization, Velocity, and Testing**

283 A clear utilization hierarchy emerged: **daily drivers** (4 agents, ~88 sessions: Developer,
284 Reviewer 1, QA, Architect), **regular contributors** (7 agents, ~65 sessions), and **insurance**
285 **policies** (5 agents, ~15 sessions: DevOps, DBA, Analytics, SRE, Doc Writer).

286 Average shipping velocity: ~8 tools per sprint, with each tool passing dual review,
287 QA testing, and (for major features) prompt testing, conversation testing, and red team
288 assessment. The **conv-sim** framework executed 858 automated checks across 78 conversation
289 turns, achieving a 94.2% pass rate. The 50 failures cluster into infrastructure-bound (latency,
290 22/50) and agent-bound (language consistency, enum leakage, routing errors, 28/50).

291 **6 Discussion**

292 **6.1 What Actually Changes When You Separate Roles**

293 The most surprising result was not that role-separated reviewers produced better output—we
294 expected that. The surprise was *how different* the output was. A reviewer with full tool
295 access behaves like a developer who happens to be reviewing: it spots an issue, patches it
296 inline, and moves on. Remove the ability to edit, and the same model produces detailed,

■ Table 8 Red Team Attack Results ($n = 30$, 6 sessions).

Category	Att.	Block	Succ.	Fixed
Prompt Injection	8	6	2	2
Data Exfiltration	6	6	0	—
Privilege Escalation	5	2	3	3
Tenant Boundary	4	4	0	—
PII Leakage	4	3	1	1
API Security	3	1	2	2
Total	30	22	8	8

297 structured feedback—not because we asked it to, but because it has nowhere else to direct
298 its effort. We call this behavioral redirection, and it turned out to be the clearest evidence
299 for why multi-agent separation matters.

300 **6.2 Why Two Reviewers Are Not Redundant**

301 Before running the dual-review experiment, we considered the possibility that two LLM
302 reviewers would simply find the same issues with different wording. The 32.8% overlap rate
303 proved otherwise. Opus consistently caught architectural concerns but overlooked security
304 boundaries. Sonnet+Codex found race conditions and cross-tenant isolation gaps that Opus
305 never flagged. All five exclusive CRITICAL findings came from the cheaper model pairing.
306 The practical takeaway: if investing in dual review, make the two reviewers as different as
307 possible.

308 **6.3 The Case for Adversarial Testing by Default**

309 Eight of thirty attacks succeeded against code that had passed architecture review, dual
310 code review, and QA testing. Three were critical severity, including an admin API with zero
311 authentication. Four stages, four opportunities, and the issue sailed through all of them.
312 Each stage did its job; the problem is that none adopt an attacker’s mindset. The red team
313 agent, with a 203-line attack playbook, costs almost nothing to run.

314 **6.4 What the Human Operator Actually Does**

315 The pipeline replaces the *execution* of code generation, review, testing, and security auditing.
316 It does not replace the *judgment* that ties those activities together. In practice, the operator
317 spends most time on resolving ambiguity (competing approaches, severity disagreements,
318 flaky tests) and writing/maintaining system prompts. The competency profile this implies is
319 closer to a technical lead than a developer.

320 **6.5 Where This Breaks Down**

321 Four failure modes recurred: context loss in manual handoff (~once per sprint), prompt drift
322 in long sessions (beyond 30 turns), stale file-path references after refactoring, and persistent
323 model-specific blind spots (e.g., Claude consistently missing timezone handling). More
324 fundamentally, this is a single project by a single developer without a controlled baseline.
325 Survivorship bias is real. What we can say is that the pipeline was feasible and produced
326 measurable quality signals.

6.6 Seven Principles That Held Up

1. Remove capabilities rather than adding instructions.
2. Make reviewers deliberately different (model, prompt focus, tool access).
3. Invest in long system prompts (100+ lines of domain knowledge per agent).
4. Standardize output formats early.
5. Include adversarial testing as a default phase.
6. Run independent stages in parallel.
7. Start small (4–5 core agents as minimum viable pipeline).

Four anti-patterns caused the most wasted effort: monolithic agents, prompt-only role enforcement, symmetric dual review, and provisioning agents before validating need.

6.7 Future Work

The most pressing engineering problem is the human orchestrator—manual handoff between phases is the pipeline’s weakest link. Agent-to-agent communication via shared artifacts would address this. Cost-tiered model routing deserves rigorous study: our data hints that different tiers may handle different quality gates optimally. The most important question we cannot answer is whether the multi-agent approach genuinely outperforms a skilled developer with a monolithic assistant; a controlled comparison would settle this. We are also curious whether the architecture transfers to other regulated domains (healthcare/HIPAA, finance/SOX, legal/contract review).

References

- 1 Moritz Beller, Alberto Bacchelli, Andy Zaidman, and Elmar Juergens. Modern code reviews in open-source projects: Which problems do they fix? In *Proc. MSR*, 2014.
- 2 Mark Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- 3 Gelei Deng et al. MasterKey: Automated jailbreaking of large language model chatbots. In *Proc. NDSS*, 2024.
- 4 Kai Greshake et al. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proc. AISec*, 2023.
- 5 Sirui Hong et al. MetaGPT: Meta programming for multi-agent collaborative framework. *arXiv:2308.00352*, 2023.
- 6 Dong Huang et al. AgentCoder: Multi-agent-based code generation with iterative testing and optimisation. *arXiv:2312.13010*, 2024.
- 7 Md Nazmus Sakib Islam et al. MapCoder: Multi-agent code generation for competitive programming. In *Proc. ACL*, 2024.
- 8 Carlos E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.
- 9 Caroline Lemieux et al. CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *Proc. ICSE*, 2023.
- 10 Raymond Li et al. StarCoder: May the source be with you! *arXiv:2305.06161*, 2023.
- 11 Zhiyu Li et al. Automating code review activities by large-scale pre-training. In *Proc. FSE*, 2022.
- 12 Junwei Liu et al. Large language model-based agents for software engineering: A survey. *arXiv:2409.02977*, 2024.
- 13 Shenao Lu et al. LLM-based code review: A systematic literature review. *arXiv:2307.12929*, 2023.
- 14 OWASP. OWASP top 10 for LLM applications, 2025.

- 372 **15** Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt.
373 Examining zero-shot vulnerability repair with large language models. In *Proc. IEEE S&P*,
374 2023.
- 375 **16** Fábio Perez and Ian Ribeiro. Ignore this title and HackAPrompt: Evaluating prompt injection
376 in large language models. *arXiv:2211.09527*, 2022.
- 377 **17** Chen Qian et al. ChatDev: Communicative agents for software development. *arXiv:2307.07924*,
378 2023.
- 379 **18** Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Proc.*
380 *NeurIPS*, 2023.
- 381 **19** Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk,
382 and Gabriele Bavota. Using pre-trained models to boost code review automation. In *Proc.*
383 *ICSE*, 2022.
- 384 **20** Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist
385 agents. *arXiv:2407.16741*, 2024.
- 386 **21** Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation.
387 *arXiv:2308.08155*, 2023.
- 388 **22** John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering.
389 *arXiv:2405.15793*, 2024.